Module 1: Docker Compose understandings

In this assignment, you will use Docker Compose to define, run, and manage a multi-container application.

You are required to:

1. Explain what Docker Compose is and why it is used.
2. Create a docker-compose.yml file to run:
   - One application service (Node.js / Python / Web app)
   - One database service (MySQL / PostgreSQL / MongoDB)

3. Configure the following in the Compose file:

   - version
   - services
   - image or build
   - ports
   - environment
   - volumes
   - depends_on
   - networks

4. Build and start the services using:
   docker compose up -d

5. Verify:

   - All containers are running
   - The application can connect to the database
   - Data persists using Docker volumes

6. Stop and remove the setup using:
   docker compose down

7. Submit the docker-compose.yml file along with a short explanation of each section.

Outcome:

By completing this task, you will understand how Docker Compose simplifies running multi-container applications and how services communicate within a Docker network.

Q1. What is Docker Compose? Why it is used?

Docker Compose is a tool for defining and running multi-container Docker applications using a single YAML file. It simplifies the process of managing complex applications with interconnected services (like a web server and a database) by automating configuration with simple commands.

Docker composed is used to simplify the management of the complex applications composed of multiple interconnected services (for example a web server, a database, a cache, and an api). Instead of using long, complex docker run commands for each container, you define the entire application stack in one human-readable YAML file, which is much easier to manage.

In my case:

The application consists of:

- A Node.js backend application
- A MongoDB database
- A shared Docker network
- Persistent storage using Docker volumes

The system is fully containerized and runs without requiring MongoDB to be installed on the host machine.

Docker Compose Configuration

The docker-compose.yml file is configured with:

version: "3.9"

It defines two main services:

Service: app (Node.js Application)

- Image:
  node:18

- Purpose: Runs the backend server
- Port Mapping:
  "3000:3000"

  This maps port 3000 of the container to port 3000 on the host machine.

Environment Variables:
DB_HOST=db
DB_PORT=27017

These variables allow the Node.js app to connect to the MongoDB container using the service name (db) as the hostname.

- Dependency:

  depends_on:
  - db

  This ensures that the MongoDB container starts before the Node.js application.

Service: db (MongoDB Database)

- image:
  mongo:6

- Port Mapping:
  "27018:27017"

  This maps:

  - Port 27017 inside the container
  - To port 27018 on the host machine

  MongoDB runs internally on 27017, but is accessed externally through 27018.

- **Volumes:**

  mongo_data:/data/db

  This ensures persistent storage of MongoDB data even if the container is stopped or removed.

Networking Configuration

A custom Docker network is defined:

```
networks:
  mynetwork:
```

Both services are connected to this network, allowing them to communicate using service names.

For example:

DB_HOST = db

Here, `db` refers to the MongoDB container within the Docker network.

Volumes Configuration

A named volume is defined:

```
volumes:
  mongo_data:
```

This volume ensures:

- Database data is not lost when containers restart
- Persistent storage across deployments
- Separation of application and database layers

How the System Works

1. Docker Compose starts the *db* (MongoDB) container.
2. Docker Compose starts the *app* (Node.js) container.
3. The Node.js app connects to MongoDB using:

   Host: *db*
   Port: *27017*

4. Users access the application through:
5. *http://localhost:3000*
6. MongoDB data is stored persistently using the *mongo_data* volume.

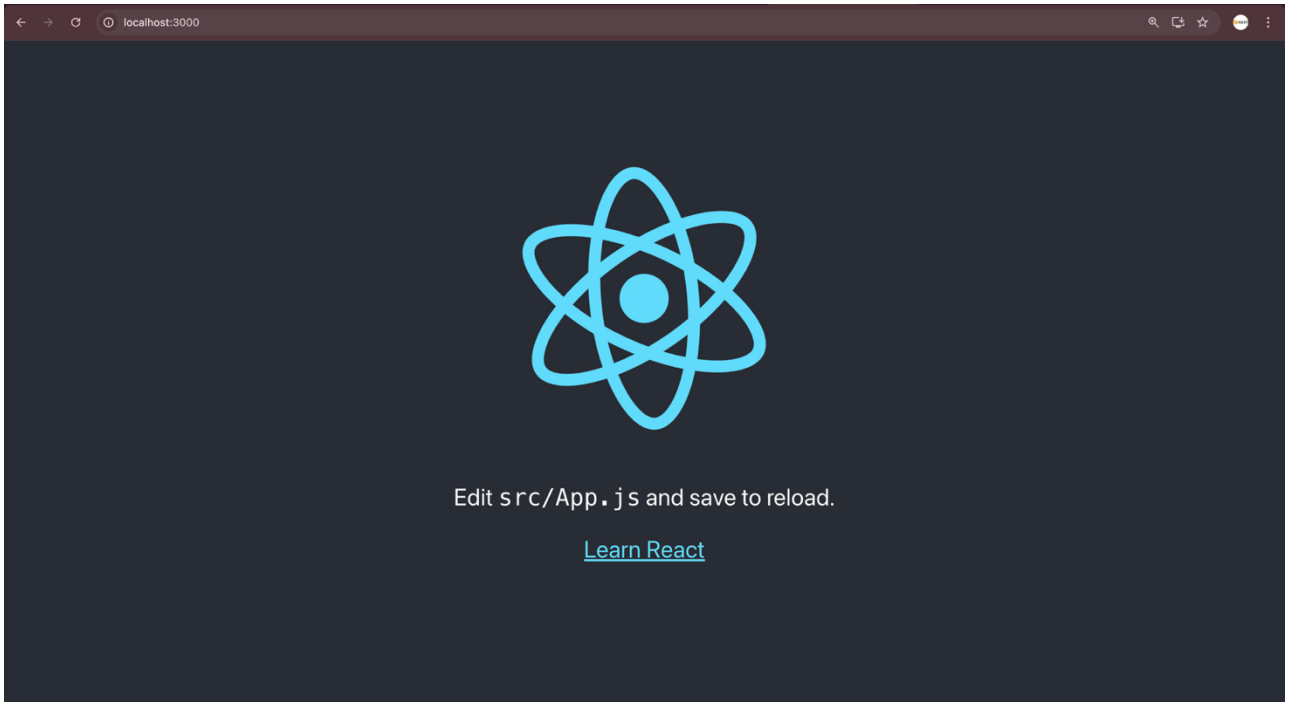
Step1:- Build and start the services using:- docker compose up -build command

```
param@Params-MacBook-Air Module 1 % docker compose up --build
 [+] Building 46.1s (21/21) FINISHED
  => [internal] load local bake definitions                                                                    0.0s
  => => reading from stdin 1.17kB                                                                              0.0s
  => [backend internal] load build definition from Dockerfile                                                  0.0s
  => => transferring dockerfile: 150B                                                                          0.0s
  => [frontend internal] load build definition from Dockerfile                                                 0.0s
  => => transferring dockerfile: 167B                                                                          0.0s
  => [frontend internal] load metadata for docker.io/library/node:18                                           1.6s
  => [auth] library/node:pull token for registry-1.docker.io                                                   0.0s
  => [backend internal] load .dockerignore                                                                     0.0s
  => => transferring context: 2B                                                                               0.0s
  => [frontend internal] load .dockerignore                                                                    0.0s
  => => transferring context: 2B                                                                               0.0s
  => [backend 1/5] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783   0.0s
  => => resolve docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783   0.0s
  => [backend internal] load build context                                                                     0.0s
  => => transferring context: 92B                                                                              0.0s
  => [frontend internal] load build context                                                                    4.4s
  => => transferring context: 236.46MB                                                                         4.4s
  => CACHED [backend 2/5] WORKDIR /app                                                                         0.0s
  => CACHED [backend 3/5] COPY package.json .                                                                  0.0s
  => CACHED [backend 4/5] RUN npm install                                                                      0.0s
  => CACHED [backend 5/5] COPY . .                                                                             0.0s
  => [backend] exporting to image                                                                              0.2s
  => => exporting layers                                                                                       0.0s
  => => exporting manifest sha256:b082c43084411f43cff5aaf13963ad3cab1b1c24ae0e39da6b3c9a6e98180183             0.0s
  => => exporting config sha256:8fef769d8aa09bb3459c0fa8c3b33bf5fc156d1dd52d6d69a87cc1831b764d75               0.0s
  => => exporting attestation manifest sha256:4e45cb017d9dcacb6d612839e488b2793cccb1ea17d90cfd4c9150c58e7d6939  0.0s
  => => exporting manifest list sha256:2644bf66dddf9fe61e1c07c2c2402c039253d166ebefe5c7bf98434aa5da7680         0.0s
  => => naming to docker.io/library/module1-backend:latest                                                     0.0s
  => => unpacking to docker.io/library/module1-backend:latest                                                  0.0s
  => [backend] resolving provenance for metadata file                                                          0.0s
  => [frontend 3/5] COPY package.json package-lock.json ./                                                     0.3s
  => [frontend 4/5] RUN npm install                                                                           13.5s
  => [frontend 5/5] COPY . .                                                                                   7.0s
  => [frontend] exporting to image                                                                            18.6s
  => => exporting layers                                                                                      12.3s
  => => exporting manifest sha256:d13cefb1cf0825e3ef09c167dd96e169641baf4d5baf28a412d9db72103621c9             0.0s
  => => exporting config sha256:6ec85739d6470ed2f4cfa54f0052dd69f0826a4f06b5c1393ebc9b4db837788c               0.0s
  => => exporting attestation manifest sha256:5f4543d6d12ee716e0c846864c0c90c7c098e477af37e2821bc09831bdbd2891  0.0s
  => => exporting manifest list sha256:269bf90d80d2c7639cead52fa1736610848d8b2cec59a5677febde4fbc1952e5         0.0s
  => => naming to docker.io/library/module1-frontend:latest                                                    0.0s
  => => unpacking to docker.io/library/module1-frontend:latest                                                 6.2s
  => [frontend] resolving provenance for metadata file                                                         0.1s
 [+] Running 7/7
  ✔ module1-backend           Built                                                                            0.0s
  ✔ module1-frontend          Built                                                                            0.0s
  ✔ Network module1_default   Created                                                                          0.0s
  ✔ Volume module1_db_data    Created                                                                          0.0s
  ✔ Container notes_db        Created                                                                          0.5s
  ✔ Container notes_backend   Created                                                                          0.1s
  ✔ Container notes_frontend  Created                                                                          0.1s
 Attaching to notes_backend, notes_db, notes_frontend
```

Step2:- Verify using :- docker ps command

```
param@Params-MacBook-Air gitlab % docker ps
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS          PORTS                                          NAMES
85e2b73cec2d   module1-frontend   "docker-entrypoint.s…"   52 seconds ago   Up 51 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   notes_f
rontend
fe9630fa4538   module1-backend    "docker-entrypoint.s…"   52 seconds ago   Up 51 seconds   5000/tcp                                       notes_b
ackend
26a3a3bf2ecb   postgres:15        "docker-entrypoint.s…"   53 seconds ago   Up 51 seconds   5432/tcp                                       notes_d
b
param@Params-MacBook-Air gitlab %
```

Step3:- Stop and remove the setup using:- docker compose down

```
param@Params-MacBook-Air Module 1 % docker compose down
[+] Running 4/4
 ✔ Container notes_frontend  Removed
 ✔ Container notes_backend   Removed
 ✔ Container notes_db        Removed
 ✔ Network module1_default   Removed
param@Params-MacBook-Air Module 1 %
```

localhost:3000